

Addendum: reproducción del Game Day de Chaos Engineering en cluster kubernetes real

MASOBAP2026 · Observabilidad U2 · Fork `jorecof/chaos_k8s` · versión 3 — Game Day completo: Experimento 1, Experimento 2 y verificación experimental de la remediación

Este addendum documenta la reproducción del laboratorio de Chaos Engineering original (diseñado para GKE) sobre un cluster kubernetes propio de 3 nodos, con datos y evidencia reales capturados durante la ejecución. La versión 3 recoge una corrida completa y automatizada del Game Day —tres experimentos encadenados, 4 077 peticiones medidas en el cliente— e incorpora dos cosas que no estaban en las versiones anteriores: un **SLI medido fuera del proceso instrumentado**, desplegado precisamente para poder ver lo que la telemetría de la aplicación no ve, y una **corrida de control que verifica la remediación propuesta** en lugar de solo enunciarla.

CORRECCIÓN RESPECTO DE LA VERSIÓN 2

La v2 explicaba que las peticiones abortadas «nunca llegan al código de la aplicación». La corrida instrumentada demuestra que **eso es falso**: el manifiesto usa `target: Response`, de modo que el corte ocurre en el camino de vuelta, después de que la aplicación procesó la petición y emitió su span. La sección 5.4 desarrolla la explicación correcta, que hace el hallazgo más severo, no menos.

1. Arquitectura real desplegada

A diferencia del docker-compose del enunciado original, el laboratorio corre sobre un cluster kubernetes de 3 VMs (VirtualBox, Debian 12 arm64, containerd 2.3.3, Calico v3.32.1, K8s v1.35). Las cargas con estado (registry, Postgres, Prometheus, Grafana, Jaeger) quedan ancladas a `kube-cp` vía `nodeSelector: rol=observabilidad`; el otel-collector corre como DaemonSet (una réplica por nodo) para que el scrape de métricas por pod nunca dependa de a cuál réplica enruta el Service. A esta topología se le añadió, para esta corrida, un **blackbox exporter** que sondea tres endpoints cada 5 s desde dentro del cluster pero fuera de los procesos instrumentados (sección 5.4).

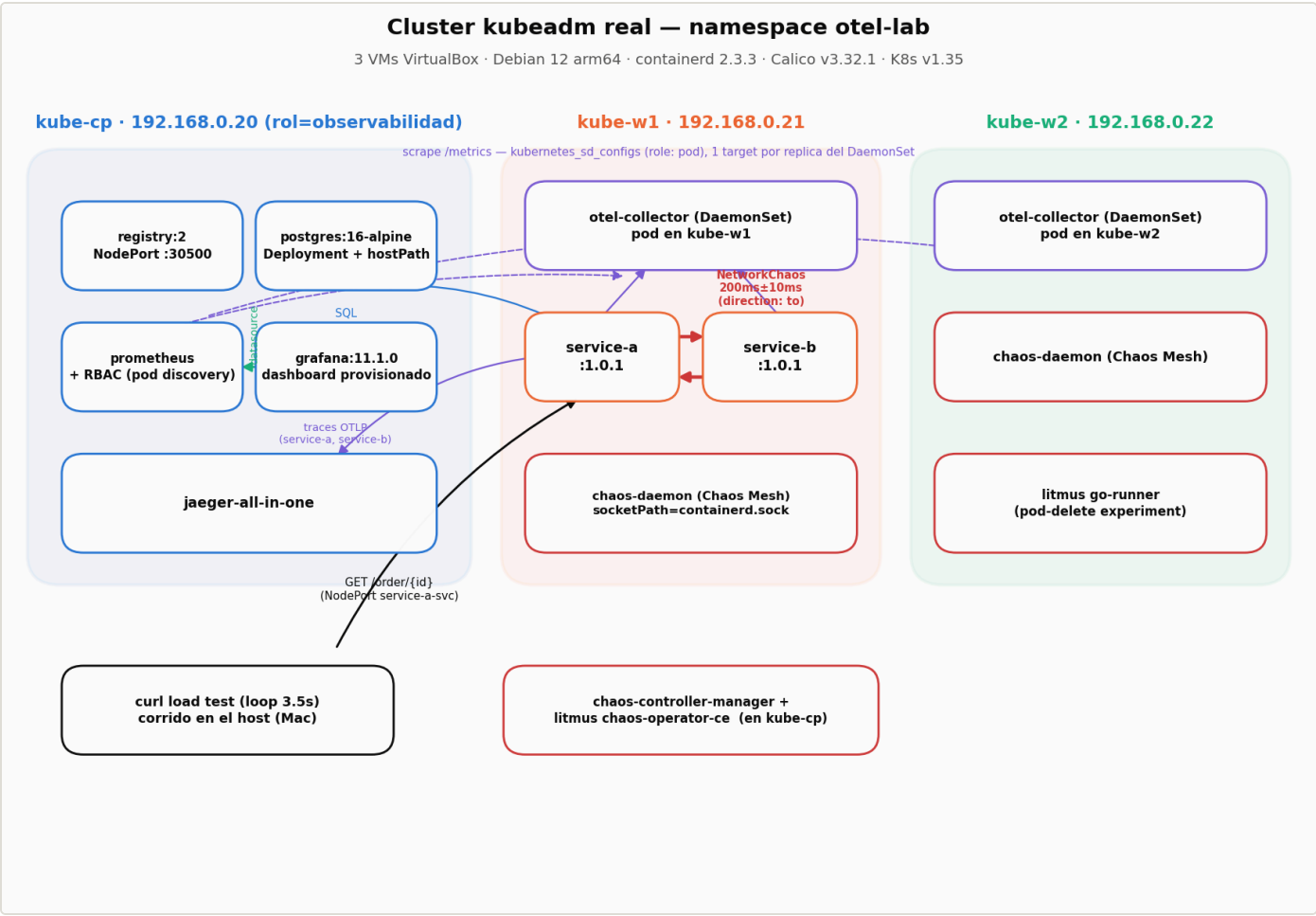


Figura 1. Topología real del cluster y flujo de datos entre componentes.

2. Verificación de estado estable (antes del chaos)

Antes de inyectar fallas se revisaron Prometheus, Jaeger y las apps para confirmar comportamiento normal. Esa revisión encontró y corrigió, sobre el sistema real, tres problemas que hubieran invalidado cualquier medición posterior:

PROBLEMA	CAUSA RAÍZ	CORRECCIÓN
Propagación de trace context rota entre service-a y service-b	<code>opentelemetry-instrumentation-fastapi >=0.48b0</code> ya no engancha con el patch global <code>instrument()</code> llamado antes de crear la app	<code>FastAPIInstrumentor.instrument_app(app, ...)</code> llamado sobre la instancia, después de crearla
Métricas de service-b ausentes o intermitentes en Prometheus	Prometheus scrapeaba el Service del otel-collector (round-robin sobre 3 réplicas DaemonSet) en vez de cada pod	<code>kubernetes_sd_configs (role: pod)</code> + RBAC nuevo (ServiceAccount + ClusterRole)
Duración de traza sin sentido en Jaeger (57m 60s)	Desfasaje de reloj entre las 3 VMs (NTP desactivado)	<code>timedatectl set-ntp true</code> + restart de <code>systemd-timesyncd</code>

A esa lista se sumó, durante la preparación de esta corrida, un cuarto problema de observabilidad: las métricas de las aplicaciones llegan a Prometheus a través del OTel Collector, de modo que su nombre real lleva el prefijo `otelcol_` y la etiqueta que identifica al servicio es `exported_job`, no `job`. Los paneles del tablero agregaban los tres servicios en una sola serie sin filtrar, con lo cual una degradación en un servicio quedaba promediada con los otros dos. Corregido antes de medir.

3. Experimento 1 — NetworkChaos (200 ms ±10 ms sobre service-b)

Chaos Mesh NetworkChaos, acción `delay`, `direction: to`, `duration: 5m` con rollback automático por TTL. Carga: cuatro workers en paralelo contra `GET /order/{id}` del NodePort de service-a (service-a → service-b → PostgreSQL), 90 s de línea base, 300 s de inyección y 240 s de observación posterior. Total 1 452 peticiones medidas en el cliente.

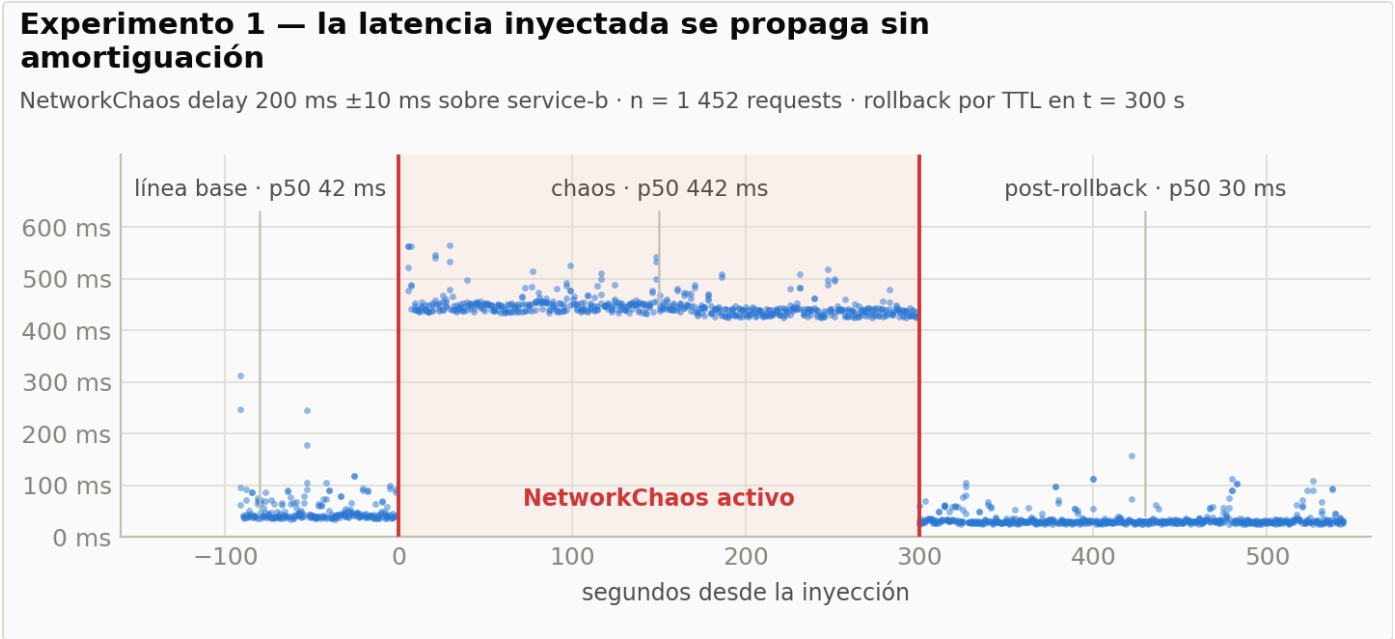


Figura 2. Latencia de cada petición. La degradación aparece y desaparece con los bordes exactos de la ventana de inyección.

FASE	N	P50	P95	P99	MÁX
Línea base	228	42,4 ms	93,6 ms	246,0 ms	313,7 ms
Chaos activo (0–300 s)	598	441,9 ms	486,1 ms	541,7 ms	565,9 ms
Post-rollback	626	30,3 ms	60,0 ms	104,1 ms	156,9 ms

La latencia añadida en la mediana es de **399,6 ms sobre 200 ms inyectados**, exactamente **2,00x**. La cifra no es casual: `direction: to` aplica el retardo en ambos sentidos del tráfico del pod, de modo que el round-trip paga el delay dos veces. El sistema no amortigua nada — propaga la degradación al usuario final en proporción directa— pero tampoco la amplifica: no hubo ni un error en las 1 452 peticiones, el throughput se mantuvo y ningún pod reinició.

El rollback por TTL funcionó de forma limpia: la latencia vuelve por debajo del doble de la línea base en **t+300,0 s**, es decir en el instante nominal, sin intervención manual. Este comportamiento es el punto de comparación que hace significativo lo que ocurre en el Experimento 2.

3.1 Dónde vive la latencia: análisis de spans

De una ejecución previa del mismo experimento se conservan dos trazas del mismo endpoint, una durante la inyección (579,89 ms) y otra después del rollback (58,99 ms), que permiten ubicar el retardo a nivel de span.

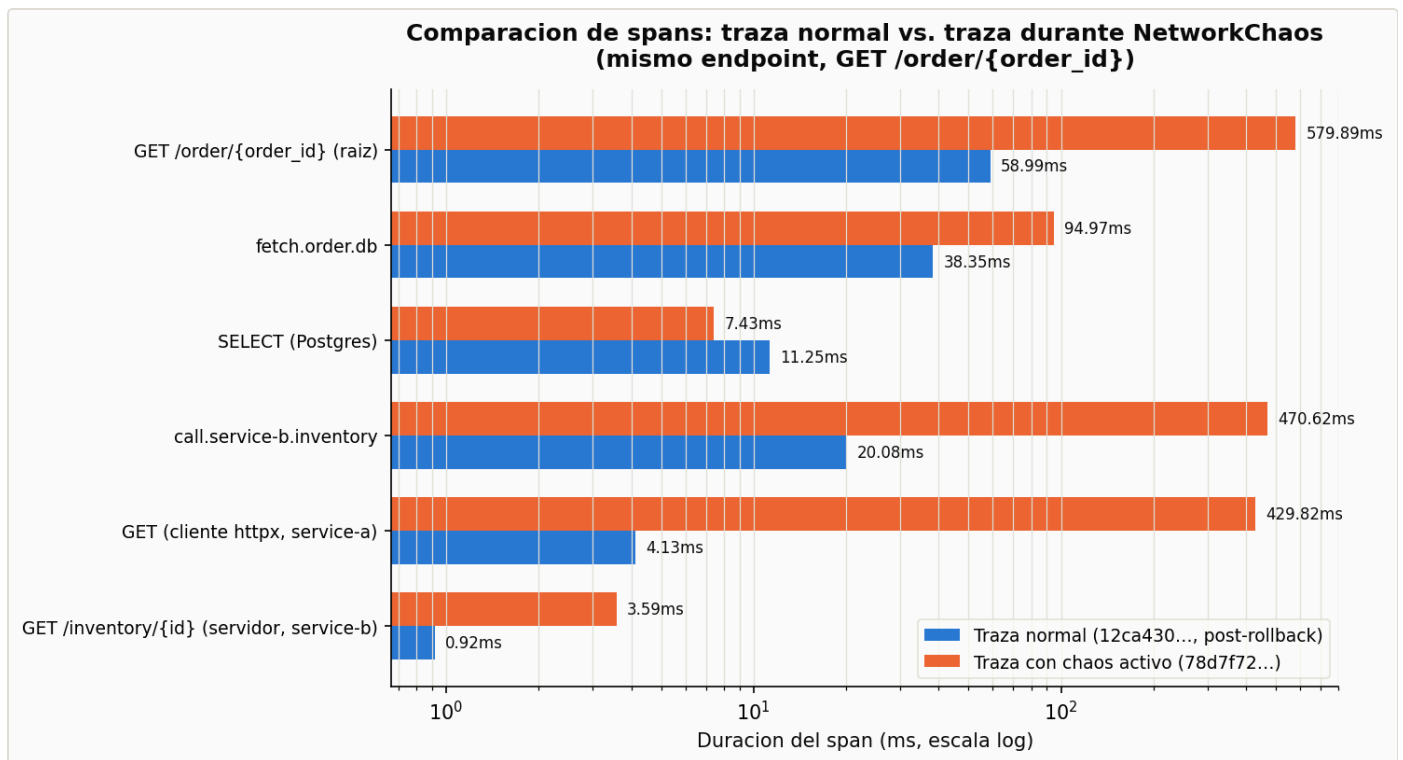


Figura 3. Duración por span, misma operación, traza normal vs. traza con chaos activo.

El 96 % del tiempo agregado se concentra en un único salto: el span cliente `GET` (httpx, dentro de service-a) pasa de 4,13 ms a 429,82 ms, mientras que el span servidor de service-b apenas crece de 0,92 ms a 3,59 ms. La diferencia entre lo que ve el cliente y lo que tarda realmente el servidor (426,2 ms) es el efecto de la falla inyectada. El resto de la traza —`SELECT` a Postgres, cache hit, spans internos— no se mueve: el blast radius quedó contenido en el salto service-a → service-b, tal como declaraba el manifiesto.

PRIMERA LECCIÓN DE OBSERVABILIDAD

El p95 propio de service-b permanece plano durante toda la inyección. Un tablero centrado en la salud de la dependencia habría estado en verde mientras el usuario final sufría un 10x de latencia. Detectar este modo de fallo exige medir extremo a extremo, no por componente.

4. Experimento 2 — HTTPChaos: error rate del 10 % en data-service

El segundo experimento inyecta fallas de aplicación en lugar de latencia de red: un `HTTPChaos` con `target: Response` y `abort: true` sobre `data-service`. La diferencia conceptual con el laboratorio Docker original importa: allí el error rate se simulaba con `if random.random() < 0.1: raise HTTPException(500)` dentro del código; aquí la aplicación no tiene ningún `random()` y la falla es completamente externa a ella. Esa diferencia es la que produce el hallazgo principal.

4.1 Correcciones al manifiesto del enunciado

- **Cuatro objetos de chaos en un solo archivo** (`HTTPChaos` + `PodChaos` + `StressChaos` + `Schedule`): un `kubectl apply` los habría inyectado todos a la vez, haciendo imposible atribuir el efecto observado a una causa. Se separó en `experiment-2-http-error.yaml` y `experiment-2-alternativas.yaml`.
- **value: "100" con mode: fixed-percent**, es decir 100 % de error, no el 10 % que anunciaban los comentarios del propio archivo.
- **La granularidad del 10 % es imposible con 2 réplicas.** `HTTPChaos` no tiene un campo de probabilidad por petición: el porcentaje se aplica a la selección de pods, y el pod seleccionado aborta el 100 % de las suyas. Con 2 réplicas el mínimo alcanzable es 50 %. Se escaló `data-service` a 10 réplicas con `value: "10"` → 1 pod → ~10 % agregado. Además `data-service-svc` pasó de `ClusterIP` a `NodePort`, porque `kubectl port-forward` fija la conexión a un solo pod y habría sesgado la medición a 0 % o 100 %.

4.2 Incidente de capacidad al escalar

INCIDENTE DURANTE LA PREPARACIÓN

Al escalar a 10 réplicas por primera vez, 5 pods se programaron en `kube-cp`, que ya corría control-plane, Postgres, registry, Prometheus, Grafana y Jaeger sobre 3,8 GB de RAM. El nodo se saturó: load average por encima de 100, apiserver OOM-killed y en crash-loop, `kubectl` respondiendo con `TLS handshake timeout`. El diagnóstico hubo que hacerlo por ssh con `crictl ps -a`, precisamente porque la API estaba caída.

Corrección: RAM de las 3 VMs aumentada (control-plane 4→8 GB, workers 2→4 GB) y `nodeAffinity` con `node-role.kubernetes.io/control-plane DoesNotExist` en el Deployment. En las corridas definitivas las 10 réplicas quedaron 5 en `kube-w1` y 5 en `kube-w2`.

Lección: el blast radius de un experimento no es solo el que declara el manifiesto de chaos. Escalar el objetivo es parte del experimento, y sin una restricción de scheduling el daño alcanzó al plano de control — el componente que uno necesita justamente para observar y revertir.

4.3 Diseño de la corrida

El Game Day se automatizó en `scripts/run-gameday.sh`, que encadena los tres experimentos sin intervención: preflight (aborta si hay objetos de chaos vivos de corridas anteriores), línea base de 90 s, inyección de 300 s, 240 s de observación posterior al rollback y limpieza garantizada, con muestreo cada 15 s del estado del CRD y de los reinicios de cada pod, y recolección de trazas de Jaeger y series de Prometheus al cierre. La carga son cuatro workers en paralelo, en circuito abierto.

CORRIDA	ÚNICO PARÁMETRO QUE CAMBIA	QUÉ PONE A PRUEBA
A	path: "*"	El experimento tal como lo define el enunciado: el blast radius incluye GET /health , o sea la livenessProbe .
B	path: "/data/*"	La remediación propuesta: el health check queda fuera del blast radius. Todo lo demás es idéntico.

4.4 Resultados de la corrida A

FASE	N	ERRORES	P50 OK	P95 OK
Línea base	232	0 — 0,0 %	14 ms	26 ms
Chaos activo (0-300 s)	524	56 — 10,7 %	12 ms	21 ms
Post-rollback (300-540 s)	411	33 — 8,0 %	12 ms	18 ms

El 10,7 % medido contra el 10 % nominal valida el diseño de la selección por pods. La fila inferior es la que no debería existir: 240 s después del duration configurado el sistema seguía fallando. La latencia de las peticiones exitosas no se degrada en ninguna fase — la falla es binaria, así que un SLI de latencia tampoco la habría detectado.

5. Hallazgo 1 — la telemetría de la aplicación afirma éxito

Durante la ventana del experimento, Jaeger registró **1 285 trazas de data-service**, todas con **http.status_code = 200** (2 233 spans, cero 4xx y cero 5xx), mientras el cliente medía 89 peticiones fallidas en esa misma ventana.

El tablero de trazas sigue ciego a la falla inyectada

Corrida A · misma ventana medida por dos fuentes · 2 233 spans exportados, ninguno con código de error

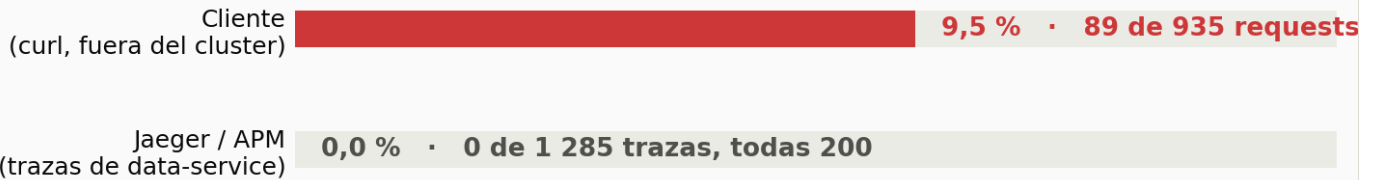


Figura 4. El mismo intervalo medido por dos fuentes de telemetría distintas.

5.1 El mecanismo: el corte ocurre después del span

La explicación no es que las peticiones no lleguen a la aplicación. El manifiesto usa `target: Response`: Chaos Mesh intercepta el **camino de vuelta**. La petición entra normalmente, la aplicación la procesa, consulta la base de datos, construye la respuesta, la marca como 200 y cierra su span — y solo entonces el proxy aborta la conexión, de modo que esos bytes nunca llegan al cliente.

La corrida B lo demuestra sin ambigüedad, porque allí las diez réplicas son estables y no hay reinicios que confundan la medición: **las diez registran el mismo tráfico**, entre 0,230 y 0,330 req/s, incluida la que tiene el chaos aplicado. La suma que contabiliza la aplicación es de **2,77 req/s** frente a las **2,59 req/s** que ofrece el cliente. No falta ni una.

La aplicación no se entera: procesa y responde 200 a lo que el cliente nunca recibe

Una línea por réplica de data-service durante la corrida B (10 con tráfico). El abort ocurre en el camino de RESPUESTA: la app ya hizo el trabajo y emitió su span cuando Chaos Mesh corta la conexión. Suma registrada por la app 2,77 req/s frente a 2,59 req/s ofrecidas por el cliente.

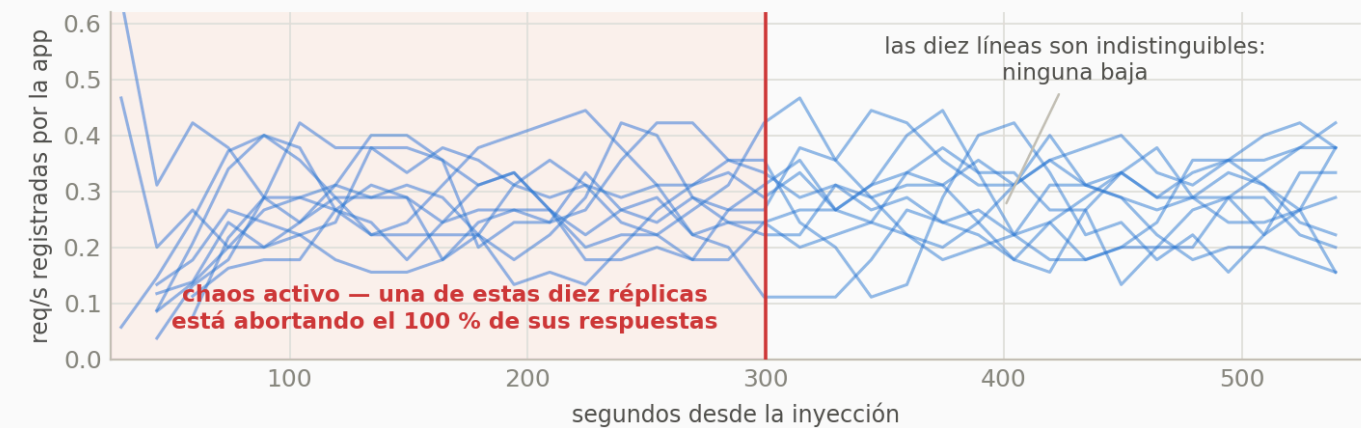


Figura 5. Una línea por réplica de data-service durante la corrida B. Una de ellas está abortando el 100 % de sus respuestas y es indistinguible de las otras nueve.

POR QUÉ ESTO ES PEOR QUE UN HUECO DE DATOS

Un tablero sin datos delata que algo pasa: el operador ve una serie que se corta y sospecha. Aquí no hay hueco. La instrumentación produce telemetría completa, puntual y **equivocada**: 1 285 trazas que afirman que todo salió bien, para un intervalo en el que uno de cada diez usuarios no recibió respuesta. El sistema de observabilidad no está callado; está dando una respuesta falsa con plena confianza.

Ninguna cantidad de instrumentación *dentro* del proceso corrige esto, porque desde el punto de vista del proceso la petición efectivamente tuvo éxito. La verdad solo existe donde está el cliente.

5.2 La remediación implementada: un SLI medido fuera del proceso

Para esta corrida se desplegó un **blackbox exporter** que sondea cada 5 s tres endpoints: `/data/products` y `/health` de data-service, y `/health` de service-a como control fuera del blast radius. Es la única fuente del tablero capaz de ver la falla, y de paso registra *cuál* endpoint falla, que resulta ser la clave del segundo hallazgo.

El SLI de borde ve la falla, y explica el crash-loop

Cada marca roja es una sonda del blackbox exporter que falló, una cada 5 s. En A el health check falla 11 veces y el kubelet mata el contenedor; en B no falla nunca y el pod sobrevive. service-a, fuera del blast radius, no se toca en ninguna de las dos.

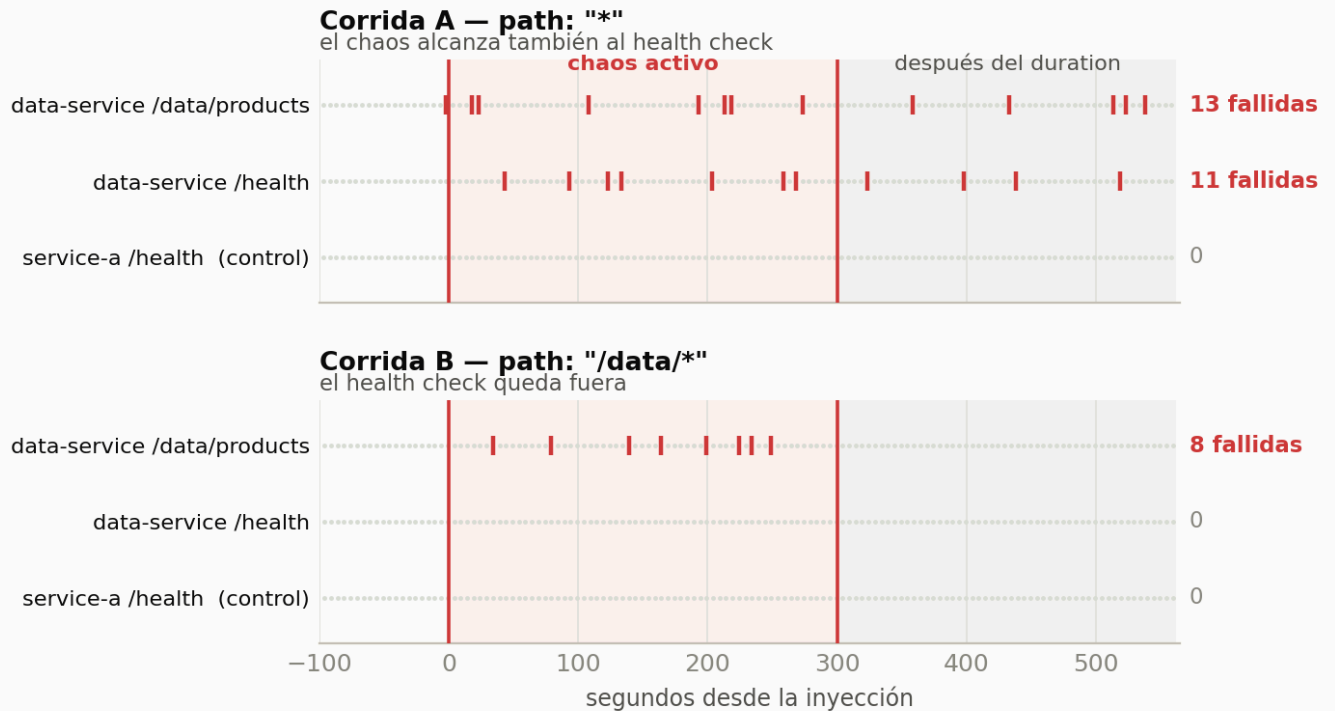


Figura 6. Cada marca es una sonda fallida. En A también falla el health check; en B no falla nunca. service-a, fuera del blast radius, no se ve afectado en ninguna de las dos corridas.

Como detalle operativo, el cliente ve `http_code = 000` —un corte de conexión, no un código HTTP—, de modo que un SLI definido como «proporción de respuestas 5xx» tampoco contaría estos fallos aunque se midiera desde fuera. La definición del indicador importa tanto como el punto donde se mide.

6. Hallazgo 2 — el rollback automático no ocurrió

A diferencia del Experimento 1, donde el TTL revirtió la falla en el instante nominal, en la corrida A el `duration: 5m` no detuvo nada: el último error se midió en **t+534,8 s**, 234,8 s más allá del fin nominal. La causa se reconstruyó cruzando el muestreo de pods con los eventos del CRD, y la figura 6 la hace visible de un vistazo: **en A también falla el health check**.



Figura 7. Cadena causal del crash-loop y del rollback fallido.

El pod objetivo acumuló **6 reinicios** durante el experimento; los otros nueve, ninguno. El intervalo entre reinicios es de 80 s constantes, exactamente lo que predice la configuración de la sonda: `livenessProbe: httpGet /health` con `periodSeconds: 20` y `failureThreshold: 3` (60 s) más la terminación del contenedor. El manifiesto declara `path: "*" ,` así que la respuesta del health check se aborta igual que la del tráfico de negocio y el kubelet concluye, correctamente desde su punto de vista, que el contenedor está muerto.

Cada reinicio destruye el network namespace del contenedor y con él las reglas de iptables que Chaos Mesh había inyectado. Cuando a los 300 s el controlador intenta revertir, el chaos-daemon ya no encuentra el tproxy al que dirigirse: el `describe` registra **18 eventos Recover / Failed** con `apply config: send http request: unexpected EOF`, todos a partir del instante exacto del fin nominal. El record quedó en `phase: Injected/Wait` con `AllRecovered: False`.

EFECTO SECUNDARIO: EL CRD NO SE DEJA BORRAR

El `finalizer` de Chaos Mesh espera a que el `recovery` termine antes de permitir el borrado del objeto. Como el `recovery` nunca puede completar, `kubectl delete httpchaos` queda colgado indefinidamente. Fue necesario forzarlo con `kubectl patch ... -p '{"metadata":{"finalizers":[]}}'`. Quitar el `finalizer` borra el objeto de Kubernetes pero **no** revierte las reglas inyectadas: lo que realmente las elimina es destruir el pod, porque se van con su `network namespace`. La secuencia correcta de limpieza manual es `finalizer` → borrar el pod objetivo → reescalar el Deployment, y así quedó implementada en el script para que ninguna corrida futura deje el cluster sucio.

7. Verificación experimental de la remediación

La remediación evidente —sacar el health check del blast radius— se propuso en la versión anterior de este documento sin comprobarla. La corrida B la pone a prueba: el mismo experimento, la misma carga y el mismo 10 % nominal, cambiando un único carácter del manifiesto.

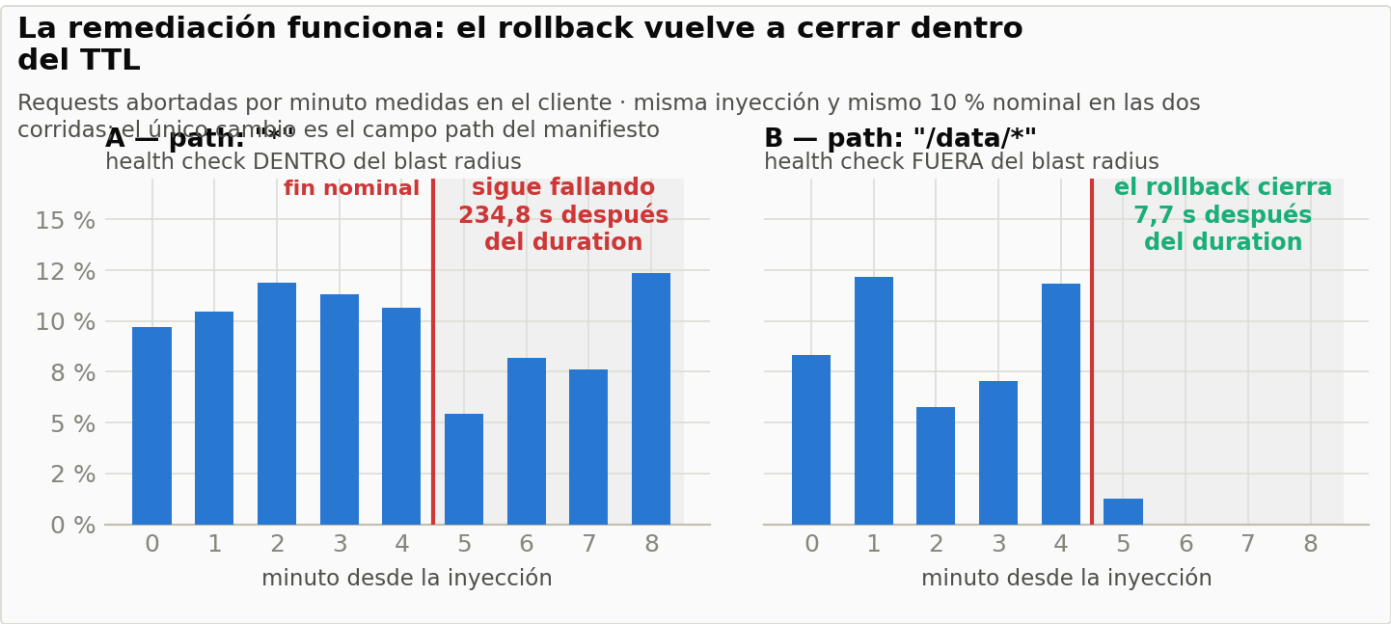


Figura 8. Requests abortadas por minuto en cada corrida. La inyección es igual de efectiva en las dos; lo que cambia es qué pasa al vencer el TTL.

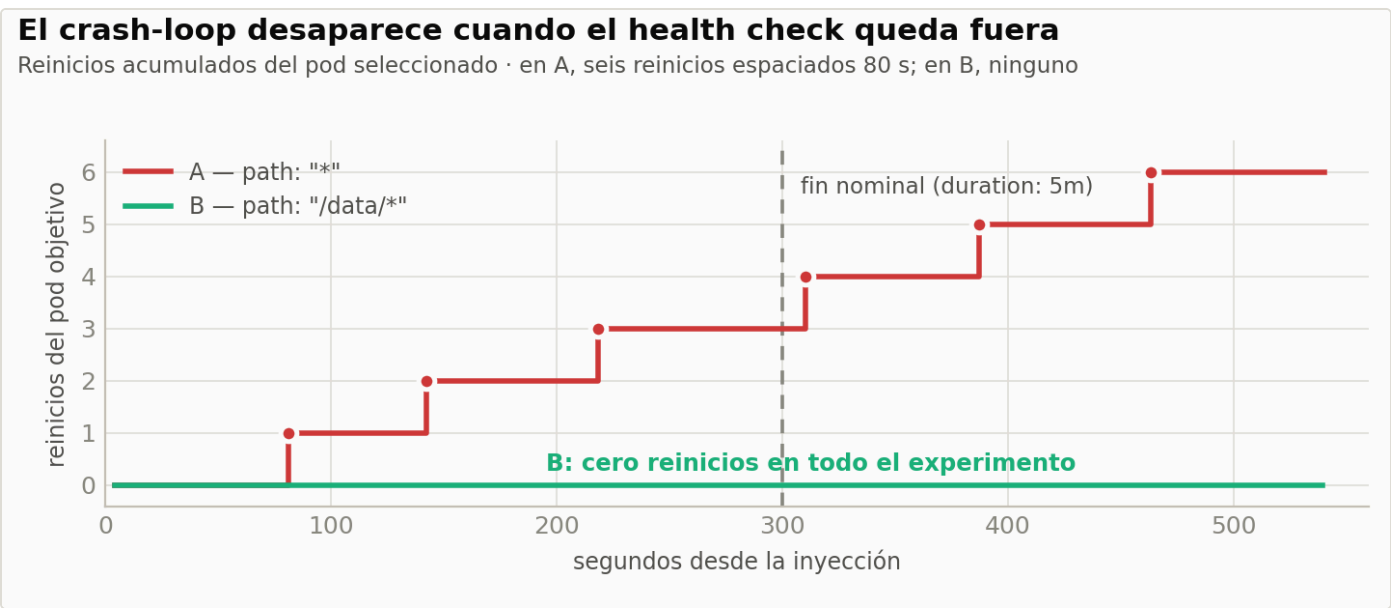


Figura 9. Reinicios acumulados del pod seleccionado.

MÉTRICA	A — PATH: "*"	B — PATH: "/DATA/*"
Error rate durante la inyección	10,7 %	9,0 %
Error rate después del TTL	8,0 %	0,3 %
Último error observado	t+534,8 s	t+307,7 s
Desfase del rollback	+234,8 s	+7,7 s
Reinicios del pod objetivo	6	0
Sondas fallidas en /health	11	0
Eventos Recover / Failed	18	0
¿Hubo que forzar el finalizer?	Sí	No
Espera del cliente antes del corte (p50)	9,86 s	13 ms

VEREDICTO

La remediación funciona. Con el health check fuera del blast radius el pod no reinicia, Chaos Mesh conserva la referencia al contenedor, el recovery se ejecuta sin un solo evento fallido y el experimento se cierra 7,7 s después del TTL — dentro del margen del propio muestreo. La inyección sigue siendo igual de efectiva: 9,0 % de error contra el 10 % nominal.

7.1 Un beneficio adicional que no se había previsto

La versión 2 de este addendum atribuía al abort un cuelgue de entre 8 y 11 segundos y lo explicaba como el timeout de retransmisión de TCP. La comparación A/B muestra que esa lectura era incorrecta.

El cuelgue de ~10 s lo causa el crash-loop, no el abort

Con el health check fuera del blast radius el fallo pasa de colgar 10 s a cortar en 13 ms: 760x más rápido



Figura 10. Tiempo que espera el cliente antes de recibir el corte, en escala logarítmica.

En la corrida A el cliente espera una mediana de **9,86 s**; en la B, **13 ms**. El cuelgue no lo produce el abort sino el crash-loop: mientras el kubelet está matando y recreando el contenedor, las conexiones que el Service enruta hacia ese pod quedan colgadas hasta agotar el timeout. Con el pod sano, el abort es un RST

inmediato. La remediación no solo restaura el rollback: convierte un fallo que bloquea al cliente diez segundos en uno que corta en trece milisegundos, **760x más rápido**. Un cliente con reintento y timeout corto sobrevive al segundo caso y no al primero.

8. Debilidades sistémicas y remediaciones

#	ACCIÓN	IMPLEMENTACIÓN Y ESTADO
1	Excluir el health check del blast radius	<code>path: "/data/*"</code> en vez de <code>path: "**"</code> , o una <code>livenessProbe</code> de tipo <code>exec / tcpSocket</code> que no atravesase el tproxy. Verificada experimentalmente en la corrida B (sección 7).
2	SLI de disponibilidad medido fuera del proceso	blackbox exporter sondeando cada 5 s, con job propio en Prometheus y dos paneles en el tablero. Implementada y en uso : es la única fuente que registró la falla.
3	Contar cortes de conexión como fallas	El 100 % de los errores llegan al cliente como <code>http_code = 000</code> , no como 5xx. El SLI debe definirse sobre <code>probe_success</code> , no sobre proporción de 5xx. Pendiente de formalizar en el SLO.
4	Restricción de scheduling para todo objetivo de chaos	<code>nodeAffinity</code> excluyendo el control-plane en el Deployment de data-service. Aplicada tras el incidente de la sección 4.2.
5	Runbook de limpieza y preflight	Secuencia finalizer → pod → réplicas, y verificación de que no quedan objetos de chaos vivos antes de la siguiente corrida. Implementada en <code>run-exp2.sh</code> y <code>run-gameday.sh</code> .
6	Corregir las queries del tablero	Prefijo <code>otelcol_</code> y desglose por <code>exported_job</code> : sin él, una degradación de un servicio queda promediada con la de los otros dos. Aplicada .
7	Presupuesto de tiempo por salto en el cliente	Timeouts de conexión y lectura por dependencia, más reintento acotado. El Experimento 1 muestra propagación 1:1 sin amortiguación alguna. Pendiente.

9. Conclusiones

La reproducción sobre infraestructura real confirma la hipótesis de estado estable y, en el Experimento 1, muestra degradación proporcional a la falla inyectada con recuperación automática en el instante nominal: los tres pilares de un Game Day —hipótesis, blast radius acotado, rollback automático— con evidencia real de Prometheus, Grafana y Jaeger.

El Experimento 2 es más valioso precisamente porque *no* se comportó como el diseño prometía, y sus dos hallazgos solo aparecen al ejecutar sobre un cluster real:

- **La telemetría de la aplicación no se quedó sin datos: registró datos falsos.** 1 285 trazas afirmando 200 OK para un intervalo con 89 peticiones fallidas. Como el corte ocurre en el camino de vuelta, después de que la aplicación cerró su span, ninguna instrumentación interna al proceso puede detectarlo. La verificación de disponibilidad tiene que vivir donde vive el cliente.
- **El rollback automático falló, y la causa raíz estaba en el propio experimento:** el chaos alcanzaba al health check, provocando un crash-loop que destruía las reglas que Chaos Mesh necesitaba para revertirse. La garantía de seguridad de un Game Day —«duración acotada, rollback automático»— es condicional, y una de sus condiciones es que el objetivo siga vivo.

La corrida de control cierra el ciclo completo: hipótesis, experimento, hallazgo, remediación y **verificación de la remediación**. Cambiar un único campo del manifiesto llevó el desfase del rollback de 234,8 s a 7,7 s, los reinicios de 6 a 0 y los eventos de recuperación fallida de 18 a 0, sin perder efectividad en la inyección. Ese paso final —comprobar que el arreglo arregla— es lo que separa un informe de incidente de un ejercicio de anti-fragilidad, y es también la razón por la que un Game Day merece automatizarse: la corrida completa que produjo toda esta evidencia son 40 minutos desatendidos y un comando.

Evidencia: `results/gameday-20260830-001312/` — 4 077 peticiones medidas en el cliente entre los tres experimentos, muestreo de pods y del CRD cada 15 s, eventos del `describe`, series de Prometheus a 5 s de resolución y trazas de Jaeger de cada ventana. Corrida reproducible con `scripts/run-gameday.sh`.